

EPAM Java Backend Rounds 2 & 3 Interview Prep Guide

Round 2: Spring Boot · Microservices · Databases · REST APIs · Design Patterns

Round 3: Security · Secure Coding · CI/CD · Architecture · Agile · React

Prepared for: Tuesday, July 21, 2026 · 6:30-8:00 PM IST · 1.5 hrs · Focus: Backend

1. Spring Boot Core & Bean Lifecycle

Q: What does @SpringBootApplication actually do?

A: It's a convenience meta-annotation combining three: @Configuration (marks the class as a source of bean definitions), @EnableAutoConfiguration (tells Spring Boot to auto-configure beans based on classpath dependencies and properties, e.g., auto-configuring a DataSource if a JDBC driver is on the classpath), and @ComponentScan (scans the package and sub-packages for @Component/@Service/@Repository/@Controller classes to register as beans).

Q: Explain the Spring Bean lifecycle briefly.

A: Container instantiates the bean -> populates properties/dependencies (DI) -> calls Aware interfaces if implemented (BeanNameAware, ApplicationContextAware) -> calls @PostConstruct method / InitializingBean.afterPropertiesSet() -> bean is ready and in use -> on container shutdown, calls @PreDestroy / DisposableBean.destroy().

```
@Component
public class CacheManager {
    @PostConstruct
    public void init() { System.out.println("Cache warmed up"); }

    @PreDestroy
    public void cleanup() { System.out.println("Cache flushed to disk"); }
}
```

Q: What is the difference between @Component, @Bean, and @Configuration?

A: @Component (and its specializations) is a class-level annotation for classes you own — Spring detects it via component scanning and manages it as a bean. @Bean is a method-level annotation used inside an @Configuration class to manually declare a bean, typically for third-party classes you don't own (e.g., configuring a RestTemplate or ObjectMapper) or when bean creation needs custom logic. @Configuration marks a class as a source of such @Bean definitions and ensures those methods are proxied so repeated calls return the same singleton instance.

```
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Q: What are Spring profiles, and why use them?

A: Profiles (@Profile("dev"), @Profile("prod")) let you register different beans or configuration depending on the active environment, activated via spring.profiles.active. Common use: a mock/in-memory PaymentGateway bean for dev and test profiles, and a real one wired to a live payment provider for prod.

Q: What is Spring AOP, and where is it commonly used in real projects?

A: AOP (Aspect-Oriented Programming) lets you inject cross-cutting behavior (logging, security checks, transaction management, performance monitoring) around method calls without modifying the method itself, using proxies. @Transactional, @Cacheable, and @Async are themselves implemented as AOP-based proxies. A common custom use case is an @Around advice that logs method entry/exit time for performance profiling.

```
@Aspect
@Component
public class LoggingAspect {
    @Around("@annotation(LogExecutionTime)")
}
```

```

    public Object logTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();
        System.out.println(pjp.getSignature() + " took " + (System.currentTimeMillis() - start) + "ms");
        return result;
    }
}

```

Q: Why doesn't @Transactional work on a private method, or on a method called from within the same class?

A: @Transactional (like most Spring annotations relying on AOP) works via proxies — Spring wraps the bean in a proxy that intercepts the annotated method call to start/commit/rollback a transaction. A private method can't be overridden by a subclass proxy, so the proxy can't intercept it. Similarly, calling an @Transactional method from another method in the same class bypasses the proxy entirely (it's a direct 'this.method()' call, not going through the proxy), so the transaction never starts. Fix: move the method to another bean and call it through that bean, or use self-injection / AspectJ compile-time weaving.

```

@Service
public class OrderService {
    @Transactional
    public void placeOrder() {
        // works correctly - called from outside via the proxy
    }

    public void wrapper() {
        placeOrder(); // BUG: bypasses the proxy, no transaction started
    }
}

```

2. Spring Data JPA & Transactions

Q: What is the difference between JPA, Hibernate, and Spring Data JPA?

A: JPA is a specification (a set of interfaces/annotations like @Entity, EntityManager) defining how Java objects map to relational tables. Hibernate is the most popular implementation of that specification, doing the actual SQL generation and ORM work. Spring Data JPA sits on top of JPA, eliminating boilerplate by auto-generating repository implementations from interface method names or @Query annotations.

```

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByCity(String city); // derived query, no SQL needed
    List<Employee> findByCityAndSalaryGreaterThan(String city, double salary);

    @Query("SELECT e FROM Employee e WHERE e.department.name = :dept")
    List<Employee> findByDepartmentName(@Param("dept") String dept);
}

```

Q: Explain @OneToMany / @ManyToOne mapping with an example (e.g., Department has many Employees).

```

@Entity
public class Department {
    @Id @GeneratedValue Long id;
    String name;
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    List<Employee> employees;
}

@Entity
public class Employee {

```

```

    @Id @GeneratedValue Long id;
    String name;
    @ManyToOne
    @JoinColumn(name = "department_id")
    Department department;
}

```

Q: What is lazy vs eager fetching, and what problem can eager fetching cause?

A: Lazy fetching loads related entities only when they're actually accessed (a proxy is returned initially); eager fetching loads them immediately along with the parent entity. @ManyToOne defaults to eager, @OneToMany defaults to lazy. Overusing eager fetching can cause performance problems by pulling in large object graphs you don't need, or the classic 'N+1 select problem' where fetching a list of parents triggers a separate query per child collection.

Q: What is the N+1 select problem, and how do you fix it?

A: It happens when you fetch N parent rows, then lazily trigger N additional queries (one per parent) to fetch each parent's related child collection, instead of one efficient join query. Fix with a JOIN FETCH in JPQL, an @EntityGraph annotation, or batch fetching configuration.

```

@Query("SELECT d FROM Department d JOIN FETCH d.employees WHERE d.id = :id")
Department findWithEmployees(@Param("id") Long id);

```

Q: What is @Transactional's default rollback behavior?

A: By default, a Spring-managed transaction rolls back only on unchecked exceptions (RuntimeException and its subclasses) or Error, not on checked exceptions. To roll back on a checked exception too, you must specify rollbackFor explicitly: @Transactional(rollbackFor = Exception.class).

Q: What are transaction isolation levels? Name them from least to most strict.

A: READ_UNCOMMITTED (can see uncommitted changes from other transactions - 'dirty reads'), READ_COMMITTED (only sees committed data, but repeated reads in the same transaction can differ - 'non-repeatable reads'; this is PostgreSQL's and most databases' default), REPEATABLE_READ (same query returns the same rows throughout the transaction, but new rows matching a WHERE clause can appear - 'phantom reads'; MySQL's default), SERIALIZABLE (fully isolated, as if transactions ran one after another - safest but slowest, most prone to lock contention).

» TIP: Interviewers love asking 'what happens if @Transactional is called from the same class' — it's a very common trick question and shows real Spring understanding, not just annotation memorization.

3. REST API Design & HTTP

Q: Design a POST endpoint to create an Employee, including validation and proper status codes.

```

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeService service;
    public EmployeeController(EmployeeService service) { this.service = service; }

    @PostMapping
    public ResponseEntity<Employee> create(@Valid @RequestBody EmployeeRequest req) {
        Employee saved = service.create(req);
        URI location = URI.create("/api/employees/" + saved.getId());
        return ResponseEntity.created(location).body(saved); // 201 Created
    }
}

```

```

public class EmployeeRequest {
    @NotBlank private String name;
    @Email private String email;
    @Positive private double salary;
    // getters/setters
}

```

Q: What HTTP status codes should a well-designed REST API use, and when?

A: 200 OK - successful GET/PUT with a response body. 201 Created - successful POST that created a resource (include Location header). 204 No Content - successful DELETE or an update with no body to return. 400 Bad Request - malformed request or validation failure. 401 Unauthorized - missing/invalid authentication. 403 Forbidden - authenticated but not permitted. 404 Not Found - resource doesn't exist. 409 Conflict - state conflict, e.g. duplicate unique field. 422 Unprocessable Entity - semantically invalid input. 500 Internal Server Error - unhandled server-side failure.

Q: PUT vs PATCH vs POST — what's the semantic difference?

A: POST creates a new resource (not idempotent — calling it twice creates two resources). PUT replaces a resource entirely at a known URI and is idempotent (calling it repeatedly with the same body yields the same end state). PATCH applies a partial update to a resource (only the fields provided change) and is not guaranteed idempotent in general, though it often is in practice.

Q: What makes an HTTP method 'idempotent', and which methods are idempotent?

A: Idempotent means making the same request multiple times produces the same result as making it once (no additional side effects on repeats). GET, PUT, DELETE, HEAD, OPTIONS are idempotent. POST and PATCH are generally not.

Q: How do you version a REST API, and what are the trade-offs?

A: Common approaches: URI versioning (/api/v1/employees - simple, visible, but clutters the URL), header-based versioning (Accept: application/vnd.company.v1+json - cleaner URLs but less discoverable/harder to test manually), and query-parameter versioning (?version=1 - simple but easy to forget/misuse). URI versioning is the most common in practice for its simplicity and cache-friendliness.

Q: What is HATEOAS?

A: Hypermedia As The Engine Of Application State — a REST maturity concept where API responses include links to related actions/resources (e.g., a GET /orders/5 response includes a link to cancel that order), so clients can navigate the API dynamically rather than hardcoding every URL structure. Rarely fully implemented in practice, but commonly asked conceptually.

4. Exception Handling in Spring Boot

Q: How do you implement global exception handling in a Spring Boot REST API?

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        ErrorResponse body = new ErrorResponse("NOT_FOUND", ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())

```

```

        .collect(Collectors.joining(", "));
    return ResponseEntity.badRequest().body(new ErrorResponse("VALIDATION_ERROR", msg));
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
    return ResponseEntity.internalServerError()
        .body(new ErrorResponse("INTERNAL_ERROR", "Something went wrong"));
}
}

```

Q: What is the difference between @ExceptionHandler, @ControllerAdvice, and @RestControllerAdvice?

A: @ExceptionHandler marks a method (in a controller or an advice class) as the handler for a specific exception type. @ControllerAdvice applies such handlers globally across all controllers and can also return views (for traditional MVC apps). @RestControllerAdvice is @ControllerAdvice + @ResponseBody combined — the returned object is automatically serialized to JSON, which is what you want for a REST API.

Q: Checked vs unchecked exceptions — how should you decide which to use for a custom business exception?

A: Checked exceptions (extends Exception) force callers to explicitly catch or declare them — good for recoverable conditions the caller should be aware of, but can clutter method signatures across layers. Unchecked exceptions (extends RuntimeException) don't need to be declared — commonly preferred for business/domain exceptions in modern Spring apps (e.g., ResourceNotFoundException extends RuntimeException) since they keep service-layer signatures clean and propagate naturally up to a global handler.

5. Database & SQL Fundamentals

Q: Explain the ACID properties of a transaction.

A: Atomicity - a transaction's operations either all succeed or all roll back, no partial state. Consistency - a transaction moves the database from one valid state to another, respecting constraints. Isolation - concurrent transactions don't interfere with each other's intermediate state (governed by isolation levels). Durability - once committed, changes survive even a crash (written to persistent storage/logs).

Q: INNER JOIN vs LEFT JOIN vs RIGHT JOIN — explain with a quick example.

A: INNER JOIN returns only rows where the join condition matches in both tables. LEFT JOIN returns all rows from the left table plus matching rows from the right (NULLs where there's no match). RIGHT JOIN is the mirror — all rows from the right table plus matches from the left.

```

SELECT e.name, d.name AS dept
FROM employee e
LEFT JOIN department d ON e.department_id = d.id;
-- returns every employee, even ones with no department (dept = NULL)

```

Q: What is a database index, and what's the trade-off of adding more of them?

A: An index is an auxiliary data structure (commonly a B-tree) that lets the database find rows matching a condition without scanning the whole table, dramatically speeding up SELECT/WHERE/JOIN/ORDER BY performance. The trade-off: every INSERT/UPDATE/DELETE must also update all relevant indexes, so more indexes slow down writes and use more storage — index only the columns you actually filter/sort/join on frequently.

Q: What is the difference between a clustered and a non-clustered index?

A: A clustered index determines the actual physical storage order of table rows — there can be only one per table (often the primary key). A non-clustered index is a separate structure that stores pointers back to the actual rows, so a table can have many of them.

Q: How would you optimize a slow SQL query?

A: Check the execution plan (EXPLAIN/EXPLAIN ANALYZE) to see if it's doing a full table scan; add an index on filtered/joined columns; avoid SELECT * (fetch only needed columns); avoid functions on indexed columns in WHERE clauses (they prevent index usage); ensure JOINS use indexed foreign keys; consider denormalization or caching for read-heavy hot paths; and paginate large result sets instead of fetching everything at once.

Q: What is database normalization, and when might you deliberately denormalize?

A: Normalization organizes tables to reduce data redundancy and avoid update anomalies, typically up to 3NF (each non-key column depends only on the primary key). Deliberate denormalization (duplicating some data) is common in read-heavy systems or reporting/analytics tables, trading some redundancy and write complexity for much faster reads by avoiding expensive joins.

6. Microservices Architecture & Patterns

Q: What is a Service Registry, and how does service discovery work (e.g., Eureka)?

A: A Service Registry (like Netflix Eureka or Consul) is a central directory where each microservice instance registers itself on startup along with its host/port, and periodically sends a 'heartbeat' to prove it's still alive. Other services (or an API Gateway) query the registry to discover available instances of a service by name instead of hardcoding IPs, and the registry removes/marks-down instances that miss heartbeats, enabling dynamic scaling and failover.

Q: What is the role of an API Gateway in a microservices architecture?

A: A single entry point that sits in front of all microservices, handling cross-cutting concerns centrally: request routing to the correct downstream service (often via the service registry), authentication/authorization, rate limiting, request/response transformation, and sometimes aggregating responses from multiple services into one. This keeps individual services simpler and clients decoupled from internal service topology.

Q: What is a Config Server, and what problem does it solve?

A: A centralized place (e.g., Spring Cloud Config Server) to store configuration for all microservices — often backed by a Git repo — so config can be changed and even refreshed at runtime without rebuilding/redeploying each service. Solves the problem of configuration scattered across dozens of services and environments (dev/test/prod) becoming unmanageable.

Q: Synchronous vs asynchronous communication between microservices — trade-offs?

A: Synchronous (REST via RestTemplate/WebClient, or Feign client) is simple and request-response is immediate, but couples the caller to the callee's availability — if the downstream service is slow or down, the caller blocks or fails too, risking cascading failures. Asynchronous (message queues/event streaming like Kafka or RabbitMQ) decouples services in time — the caller publishes an event and moves on, consumers process it whenever they're ready — improving resilience and scalability, at the cost of eventual consistency and more complex debugging/tracing.

Q: What is the Circuit Breaker pattern, and how does Resilience4j implement it?

A: A Circuit Breaker prevents a service from repeatedly calling a downstream dependency that's already failing, which would waste resources and worsen cascading failure. It has three states: CLOSED (calls pass through normally, failures are counted), OPEN (after failures exceed a threshold, all calls immediately fail-fast without

even trying the downstream call, for a configured wait duration), and HALF_OPEN (after the wait duration, a limited number of trial calls are allowed through to test if the dependency has recovered, closing the circuit again if they succeed).

```
@Service
public class InventoryClient {

    @CircuitBreaker(name = "inventoryService", fallbackMethod = "fallback")
    public InventoryResponse checkStock(String sku) {
        return restTemplate.getForObject("/inventory/" + sku, InventoryResponse.class);
    }

    public InventoryResponse fallback(String sku, Throwable t) {
        return InventoryResponse.unavailable(sku); // graceful degradation
    }
}
```

Q: What is the Saga pattern, and why is it needed in microservices?

A: Since each microservice typically owns its own database, a single ACID transaction can't span multiple services (e.g., an Order service and a Payment service). Saga breaks a business transaction into a sequence of local transactions, each in a different service, coordinated either via Choreography (each service publishes an event that triggers the next step, no central coordinator) or Orchestration (a central saga orchestrator explicitly tells each service what to do next). If a step fails, previously completed steps are undone via compensating transactions (e.g., refund a payment) instead of a database rollback.

Q: What's a drawback of the Saga pattern?

A: It trades strong consistency for eventual consistency — there's a window where different services see different states of the overall business transaction. Compensating transactions also add significant complexity (you must design an 'undo' for every step), and debugging/tracing a distributed, multi-step saga is harder than debugging a single database transaction.

Q: What is CQRS (Command Query Responsibility Segregation)?

A: CQRS separates the write model (Commands - create/update/delete, optimized for validation and consistency) from the read model (Queries - optimized for fast, denormalized reads, often a different data store or schema entirely). Useful when read and write workloads have very different scaling/performance needs — e.g., writes go through a normalized relational DB while reads are served from a denormalized, heavily indexed or cached view.

Q: What is Two-Phase Commit (2PC), and why is it rarely used in microservices?

A: 2PC is a distributed transaction protocol: a coordinator asks all participants to 'prepare' (lock resources, vote yes/no), and only if everyone votes yes does it tell them all to 'commit'. It gives strong consistency but requires holding locks across all services for the whole duration, which severely hurts availability and throughput, and if the coordinator crashes mid-protocol, participants can be left blocked indefinitely. Most microservice architectures prefer Saga's eventual consistency over 2PC's blocking behavior.

Q: What is a correlation ID, and why is it critical in microservices logging?

A: A unique identifier generated when a request first enters the system (often at the API Gateway) and passed along through every downstream service call (via an HTTP header). Since each microservice logs independently, a correlation ID lets you filter a centralized log store (e.g., Elasticsearch/Kibana) for that ID and reconstruct the full path of a single request across all services — essential for debugging distributed failures.

7. Caching

Q: How does @Cacheable work in Spring, and what's a real use case?

A: @Cacheable wraps a method with a cache-aside proxy: on the first call with a given set of arguments, it executes the method and stores the result in the configured cache (e.g., Redis, Caffeine, or the simple in-memory ConcurrentMapCache); subsequent calls with the same arguments return the cached value without executing the method body at all. Common use case: caching a frequently-read, rarely-changed lookup like product details or country/state reference data.

```
@Service
public class ProductService {
    @Cacheable(value = "products", key = "#id")
    public Product getProduct(Long id) {
        return productRepository.findById(id).orElseThrow();
    }

    @CacheEvict(value = "products", key = "#product.id")
    public void updateProduct(Product product) {
        productRepository.save(product); // invalidate stale cache entry
    }
}
```

Q: What is cache eviction, and what strategies exist (e.g., LRU)?

A: Eviction is removing entries from a cache when it's full or entries become stale. LRU (Least Recently Used) evicts the entry that hasn't been accessed in the longest time — a common default because recently-used data is often likely to be used again soon. Other strategies: LFU (Least Frequently Used), FIFO, and TTL-based expiry (evict after a fixed time regardless of usage), often combined in real systems like Redis.

Q: What's the difference between a write-through and write-behind (write-back) cache?

A: Write-through writes to the cache and the underlying database synchronously in the same operation — simpler and consistent, but as slow as the database write. Write-behind (write-back) writes to the cache immediately and asynchronously persists to the database later — much faster for the caller, but risks data loss if the cache fails before the write is flushed.

Q: What is cache invalidation, and why is it considered a hard problem?

A: Invalidation means removing or updating a cached value when the underlying data changes, so clients don't read stale data. It's hard because in a distributed system, multiple services/instances may cache the same data independently, and there's no single moment everyone agrees the data changed — leading to race conditions between an update and a stale read, or forgotten invalidation paths causing subtly wrong data for a long time.

8. Security Basics (Spring Security / JWT)

Q: How does JWT-based authentication work in a Spring Boot REST API?

A: On login, the server validates credentials and issues a signed JWT (containing claims like user id, roles, and expiry) back to the client. The client sends this token in the Authorization: Bearer header on every subsequent request. A filter on the server verifies the token's signature and expiry (no database lookup needed, since the token is self-contained/stateless) and, if valid, sets the authenticated user in the Spring Security context for that request.

Q: What is the difference between authentication and authorization?

A: Authentication verifies who you are (login, validating credentials/tokens). Authorization determines what you're allowed to do once authenticated (role/permission checks, e.g., only ADMIN can delete a resource). A request can be authenticated but still be forbidden (403) if it fails authorization.

Q: What is the difference between JWT and a traditional session-based approach?

A: Session-based auth stores session state server-side (often in memory or a shared store like Redis) and gives the client an opaque session ID cookie — the server must look up the session on every request. JWT is stateless: all the needed info is encoded and signed in the token itself, so any server instance can validate it without shared session storage, which scales more easily across stateless microservices — but revoking a JWT before it expires is harder since there's no central session to simply delete.

Q: What is OAuth2, briefly, and how does it differ from JWT?

A: OAuth2 is an authorization framework/protocol defining how a user can grant a third-party application limited access to their resources without sharing their password (e.g., 'Sign in with Google'), using flows like Authorization Code. JWT is just a token format/structure — OAuth2 commonly uses JWTs as its access tokens, but JWT itself doesn't define the authorization flow; they solve different, complementary problems.

9. Design Patterns Frequently Asked

Q: Singleton pattern — how do you implement a thread-safe one in plain Java (without Spring)?

```
public class ConfigManager {
    private static volatile ConfigManager instance;
    private ConfigManager() {}

    public static ConfigManager getInstance() {
        if (instance == null) {
            synchronized (ConfigManager.class) {
                if (instance == null) { // double-checked locking
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }
}
```

Q: Strategy pattern — give a real backend example.

A: Strategy defines a family of interchangeable algorithms behind a common interface, selected at runtime. Real example: a `PaymentStrategy` interface with `CreditCardPayment`, `PayPalPayment`, and `UpiPayment` implementations — the checkout service picks the right one based on the user's chosen payment method, without an if/else chain scattered through business logic.

```
public interface PaymentStrategy {
    void pay(double amount);
}

public class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) { System.out.println("Paid " + amount + " via card"); }
}

public class CheckoutService {
    public void checkout(PaymentStrategy strategy, double amount) {
        strategy.pay(amount); // behavior chosen at runtime, no if/else needed
    }
}
```

Q: Factory pattern vs Builder pattern — when to use which?

A: Factory pattern centralizes object creation logic, especially when the exact subtype to create depends on some input (e.g., `NotificationFactory.create("EMAIL")` returns an `EmailNotification`). Builder pattern is for constructing a single complex object step-by-step, especially one with many optional fields, avoiding telescoping constructors — e.g., building an HTTP request or a complex configuration object fluently.

// Builder example

```
Employee emp = Employee.builder()
    .name("Alice")
    .city("Pune")
    .salary(75000)
    .build();
```

Q: What design pattern does Spring's Dependency Injection rely on?

A: It's a direct implementation of the Dependency Inversion Principle combined with the Factory pattern (the ApplicationContext acts as a factory that creates and wires beans) and, for many proxied features like @Transactional, the Proxy pattern (a runtime-generated proxy wraps the real bean to add behavior transparently).

10. Testing (JUnit & Mockito)

Q: How do you unit test a service class with a mocked repository using Mockito?

```
@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {

    @Mock
    private EmployeeRepository repository;

    @InjectMocks
    private EmployeeService service;

    @Test
    void shouldReturnEmployeeWhenFound() {
        Employee emp = new Employee(1L, "Alice");
        when(repository.findById(1L)).thenReturn(Optional.of(emp));

        Employee result = service.getEmployee(1L);

        assertEquals("Alice", result.getName());
        verify(repository, times(1)).findById(1L);
    }
}
```

Q: What's the difference between @Mock, @InjectMocks, and @Spy in Mockito?

A: @Mock creates a fully fake object where all methods return default values unless explicitly stubbed with when(). @InjectMocks creates a real instance of the class under test and injects the @Mock fields into it (via constructor, setter, or field injection). @Spy wraps a real object, calling its actual methods by default, but lets you selectively stub specific methods when needed — useful for partial mocking.

Q: What is the difference between a unit test and an integration test in a Spring Boot context?

A: A unit test isolates a single class (usually a service) with all its dependencies mocked, runs fast, and doesn't start the Spring context. An integration test (e.g., using @SpringBootTest) loads some or all of the actual Spring application context, often hitting a real (or test/embedded) database, to verify components work correctly together — slower but catches wiring and configuration issues unit tests can't.

```
@SpringBootTest
@AutoConfigureMockMvc
class EmployeeControllerIntegrationTest {

    @Autowired MockMvc mockMvc;

    @Test
    void shouldReturn201onCreate() throws Exception {
        mockMvc.perform(post("/api/employees"))
```

```

        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"name\":\"Alice\",\"email\":\"a@x.com\",\"salary\":50000}"))
        .andExpect(status().isCreated());
    }
}

```

11. Java 8 Collections & Streams — Backend-Flavored Deep Dive

Q: Given a List, group orders by customerId and sum their totalAmount per customer.

```

Map<Long, Double> totalByCustomer = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getCustomerId,
        Collectors.summingDouble(Order::getTotalAmount)));

```

Q: Group a List by status, and for each group, get just the order IDs.

```

Map<String, List<Long>> idsByStatus = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getStatus,
        Collectors.mapping(Order::getId, Collectors.toList())));

```

Q: From a List, build a Map> sorted so each department's employees are ordered by salary descending.

```

Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

byDept.forEach((dept, list) ->
    list.sort(Comparator.comparing(Employee::getSalary).reversed()));

// or fully in one stream pipeline using a downstream collector + sorted source:
Map<String, List<Employee>> byDeptSorted = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .collect(Collectors.groupingBy(Employee::getDepartment, LinkedHashMap::new, Collectors.toList()));

```

Q: What's the difference between Collectors.toMap() with 2 args vs 3 args, and why does it matter for API data?

A: The 2-arg toMap(keyMapper, valueMapper) throws IllegalStateException if two elements produce the same key — common when converting API response lists that might have duplicate IDs. The 3-arg version adds a merge function (keyMapper, valueMapper, mergeFunction) telling the collector what to do on a key collision, e.g., keep the first, keep the latest, or combine values — essential when dealing with real-world, not-guaranteed-unique API data.

```

Map<Long, Product> latestBySku = apiResponseList.stream()
    .collect(Collectors.toMap(
        Product::getSku,
        p -> p,
        (existing, replacement) -> replacement)); // keep the last one seen on collision

```

Q: How would you merge two Maps of the same key type, summing values on key conflicts?

```

Map<String, Integer> merged = new HashMap<>(mapA);
mapB.forEach((key, value) -> merged.merge(key, value, Integer::sum));

```

Q: How do you paginate a List in-memory using streams (skip/limit), e.g. for a simple API pagination layer?

```

int pageNumber = 2, pageSize = 10; // zero-indexed page
List<Employee> page = employees.stream()

```

```

        .skip((long) pageNumber * pageSize)
        .limit(pageSize)
        .collect(Collectors.toList());

```

Q: What is `Collectors.teeing()`, and when might it help in a backend aggregation scenario?

A: `teeing()` (Java 12+) applies two different downstream collectors to the same stream in a single pass and merges their results with a `BiFunction` — e.g., computing both the sum and the count of order amounts in one pass, instead of iterating the stream twice.

```

record Summary(double total, long count) {}

Summary summary = orders.stream()
    .collect(Collectors.teeing(
        Collectors.summingDouble(Order::getTotalAmount),
        Collectors.counting(),
        Summary::new));

```

Q: How would you validate/filter a stream of incoming API request objects and separate valid from invalid ones?

```

Map<Boolean, List<EmployeeRequest>> partitioned = requests.stream()
    .collect(Collectors.partitioningBy(r ->
        r.getName() != null && !r.getName().isBlank() && r.getSalary() > 0));

List<EmployeeRequest> valid = partitioned.get(true);
List<EmployeeRequest> invalid = partitioned.get(false);

```

12. Coding: Fetching & Manipulating JSON from REST APIs

Q: How do you call an external REST API from a Spring Boot service using `RestTemplate` and map the JSON response to a Java object?

```

public class WeatherResponse {
    private String city;
    private double temperature;
    // getters/setters
}

@Service
public class WeatherClient {
    private final RestTemplate restTemplate;

    public WeatherClient(RestTemplate restTemplate) { this.restTemplate = restTemplate; }

    public WeatherResponse getWeather(String city) {
        String url = "https://api.example.com/weather?city=" + city;
        return restTemplate.getForObject(url, WeatherResponse.class); // Jackson auto-deserializes JSON ->
    }
}

```

Q: Same call, but using the modern non-blocking `WebClient` (preferred over `RestTemplate` in newer Spring versions).

```

@Service
public class WeatherClient {
    private final WebClient webClient;

    public WeatherClient(WebClient.Builder builder) {
        this.webClient = builder.baseUrl("https://api.example.com").build();
    }

    public WeatherResponse getWeather(String city) {
        return webClient.get()

```

```

        .uri(uriBuilder -> uriBuilder.path("/weather").queryParams("city", city).build())
        .retrieve()
        .bodyToMono(WeatherResponse.class)
        .block(); // .block() only for simplicity here; reactive code would chain further instead}

```

Q: How do you fetch a JSON array from an API and extract a specific field from every item, e.g., all product names above a price threshold?

```

public List<String> getExpensiveProductNames(double minPrice) {
    Product[] products = restTemplate.getForObject("https://api.example.com/products", Product[].class);

    return Arrays.stream(products)
        .filter(p -> p.getPrice() > minPrice)
        .map(Product::getName)
        .collect(Collectors.toList());
}

```

Q: How do you parse a JSON response when you don't have (or don't want) a matching POJO, using Jackson's JsonNode/ObjectMapper directly?

```

ObjectMapper mapper = new ObjectMapper();
String json = restTemplate.getForObject(url, String.class);

JsonNode root = mapper.readTree(json);
String city = root.path("city").asText();
double temp = root.path("temperature").asDouble();

// Navigating a nested array inside the JSON
JsonNode items = root.path("items");
for (JsonNode item : items) {
    System.out.println(item.path("name").asText());
}

```

Q: How do you handle errors gracefully when calling an external API (e.g., it returns a 404 or times out)?

```

public WeatherResponse getWeatherSafely(String city) {
    try {
        return restTemplate.getForObject(url, WeatherResponse.class);
    } catch (HttpClientErrorException.NotFound ex) {
        throw new ResourceNotFoundException("No weather data for " + city);
    } catch (RestClientException ex) {
        // covers timeouts, connection refused, 5xx errors, etc.
        throw new ExternalServiceException("Weather service unavailable", ex);
    }
}

```

Q: How would you set a connection/read timeout when calling an external API, to avoid hanging forever?

```

@Bean
public RestTemplate restTemplate() {
    HttpComponentsClientHttpRequestFactory factory = new HttpComponentsClientHttpRequestFactory();
    factory.setConnectTimeout(3000); // ms
    factory.setReadTimeout(5000);
    return new RestTemplate(factory);
}

```

Q: How do you transform a JSON payload's structure before returning it to your own API's client (e.g., combine two external API calls into one custom DTO)?

```

public OrderSummaryDto getOrderSummary(Long orderId) {
    Order order = orderClient.getOrder(orderId); // call service A
    Customer customer = customerClient.getCustomer(order.getCustomerId()); // call service B
}

```

```

        return new OrderSummaryDto(
            order.getId(),
            order.getTotalAmount(),
            customer.getName(),
            customer.getEmail()
        );
    }
}

```

Q: How do you POST a JSON body to an external API and read back the created resource?

```

public Product createProduct(ProductRequest request) {
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    HttpEntity<ProductRequest> entity = new HttpEntity<>(request, headers);

    ResponseEntity<Product> response = restTemplate.postForEntity(
        "https://api.example.com/products", entity, Product.class);

    return response.getBody();
}

```

13. Pseudocode Practice Likely for a Backend Coding Round

Q: Pseudocode: design a simple in-memory rate limiter (fixed window) for an API endpoint.

```

FUNCTION isAllowed(userId):
    now = currentTimeInSeconds()
    windowStart = now - (now MOD WINDOW_SIZE)    // e.g., WINDOW_SIZE = 60 sec

    record = requestCounts.get(userId)
    IF record IS NULL OR record.windowStart != windowStart:
        record = { windowStart: windowStart, count: 0 }
        requestCounts.put(userId, record)

    IF record.count >= MAX_REQUESTS_PER_WINDOW:
        RETURN false    // reject - 429 Too Many Requests
    ELSE:
        record.count += 1
        RETURN true

```

Q: Real Java: same fixed-window rate limiter using ConcurrentHashMap for thread safety.

```

public class RateLimiter {
    private final int maxRequests;
    private final long windowSizeMs;
    private final Map<String, Window> counters = new ConcurrentHashMap<>();

    record Window(long windowStart, AtomicInteger count) {}

    public boolean isAllowed(String userId) {
        long now = System.currentTimeMillis();
        long windowStart = now - (now % windowSizeMs);

        Window w = counters.compute(userId, (key, existing) -> {
            if (existing == null || existing.windowStart() != windowStart) {
                return new Window(windowStart, new AtomicInteger(0));
            }
            return existing;
        });

        return w.count().incrementAndGet() <= maxRequests;
    }
}

```

```
}
```

Q: Pseudocode: design an order-processing state machine (draft -> submitted -> approved -> shipped / cancelled).

```
ENUM OrderStatus: DRAFT, SUBMITTED, APPROVED, SHIPPED, CANCELLED
```

```
FUNCTION transition(order, newStatus):
    validTransitions = {
        DRAFT: [SUBMITTED, CANCELLED],
        SUBMITTED: [APPROVED, CANCELLED],
        APPROVED: [SHIPPED, CANCELLED],
        SHIPPED: [],
        CANCELLED: []
    }

    IF newStatus NOT IN validTransitions[order.status]:
        THROW InvalidTransitionException(order.status, newStatus)

    order.status = newStatus
    saveOrder(order)
    publishEvent(OrderStatusChangedEvent(order.id, newStatus)) // e.g., to Kafka for other services
```

Q: Pseudocode: implement a simple LRU cache (common backend/coding-round staple).

```
CLASS LRUCache(capacity):
    map = OrderedMap() // preserves insertion/access order

    FUNCTION get(key):
        IF key NOT IN map: RETURN -1
        value = map.remove(key)
        map.put(key, value) // move to "most recently used" end
        RETURN value

    FUNCTION put(key, value):
        IF key IN map: map.remove(key)
        ELSE IF map.size() == capacity:
            evict the LEAST recently used entry (front of the map)
        map.put(key, value) // insert at "most recently used" end
```

Q: Real Java: LRU cache using LinkedHashMap's built-in access-order support.

```
public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // true = access-order mode
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity; // auto-evict oldest when over capacity
    }
}
```

Q: Pseudocode: retry an external API call with exponential backoff.

```
FUNCTION callWithRetry(request, maxRetries):
    attempt = 0
    delay = INITIAL_DELAY_MS

    WHILE attempt < maxRetries:
        TRY:
```

```
    RETURN callExternalApi(request)
CATCH TransientException:
    attempt += 1
    IF attempt == maxRetries: RETHROW
    sleep(delay)
    delay = delay * 2    // exponential backoff
```

Q: Pseudocode: deduplicate and merge paginated API results (e.g., calling a paged endpoint until it's exhausted).

```
FUNCTION fetchAllPages(apiEndpoint):
    results = []
    seenIds = emptySet()
    page = 0

    LOOP:
        response = callApi(apiEndpoint, page)
        FOR item IN response.items:
            IF item.id NOT IN seenIds:
                seenIds.add(item.id)
                results.add(item)

        IF response.items IS EMPTY OR NOT response.hasNextPage:
            BREAK
        page += 1

    RETURN results
```

» TIP: For the live coding portion, expect either a stream-based data transformation task (grouping/filtering API-shaped data) or a small system-design-flavored coding problem like rate limiting, LRU cache, or a state machine — practice narrating your approach out loud before typing, interviewers weight thought process heavily.

Round 3 Prep

Spring Security & Advanced, Secure Coding, CI/CD,
Architecture & Design, Agile, and React Fundamentals
Prepared for: Saturday, July 25, 2026 · 4:00-5:30 PM IST · 1.5 hrs

15. Spring Security & Advanced Spring / Spring Cloud

Q: How does the Spring Security filter chain work, at a high level?

A: Every incoming request passes through a chain of servlet filters registered by Spring Security before it ever reaches your controller — e.g., a filter that extracts and validates a JWT, a filter that checks CSRF tokens, an authorization filter that checks role/permission against the requested URL pattern. If any filter rejects the request (bad token, insufficient role), it short-circuits with a 401/403 before the controller code ever runs. You configure this chain via a `SecurityFilterChain` bean.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // often disabled for stateless REST APIs
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

Q: Method-level security with `@PreAuthorize` — how and why would you use it instead of only URL-based rules?

A: `@PreAuthorize` lets you express authorization at the method level using SpEL expressions, which is useful when the rule depends on more than just the URL — e.g., a user can only update their own profile, not just any profile under `/api/users/**`. It runs before the method executes (via an AOP proxy, same mechanism as `@Transactional`).

```
@PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
@PutMapping("/api/users/{userId}")
public ResponseEntity<User> updateUser(@PathVariable Long userId, @RequestBody UserRequest req) {
    // only admins, or the user themselves, can reach this
}
```

Q: What is CSRF, and why is it usually disabled for stateless REST APIs but not for traditional web apps?

A: CSRF (Cross-Site Request Forgery) tricks a logged-in user's browser into submitting an unwanted request to your server, exploiting the fact that browsers auto-attach cookies. It's a real risk when auth relies on session cookies, since the browser will silently send the cookie along with a forged request from another site. It's generally safe to disable for stateless, token-based (JWT in an Authorization header) APIs, because the token isn't automatically attached by the browser the way a cookie is — the attacking page has no way to read or attach it itself.

Q: What is a Spring Cloud Contract, and what problem does it solve between microservices?

A: Spring Cloud Contract lets a service provider define a contract (expected request/response pairs) that's used to auto-generate both a provider-side test (to verify the real implementation matches the contract) and a consumer-side stub (a fake server the consumer's tests can run against). This solves the classic microservices integration-testing problem: consumer teams don't need the real provider service running to test against it, and if the provider changes its API in a breaking way, the provider's own contract test fails immediately, catching the break before it reaches production.

Q: Feign Client — what is it, and how does it simplify calling other microservices compared to `RestTemplate`?

A: Feign is a declarative HTTP client — you define an interface with annotations describing the target endpoint, and Spring generates the implementation at runtime, so calling another microservice looks like calling a local method. It integrates with the service registry (Eureka) and load balancing automatically, and is generally considered cleaner and less boilerplate-heavy than manually building RestTemplate calls.

```
@FeignClient(name = "payment-service")
public interface PaymentClient {
    @PostMapping("/api/payments")
    PaymentResponse processPayment(@RequestBody PaymentRequest request);
}

// Usage elsewhere - looks like a normal method call
@Service
public class OrderService {
    private final PaymentClient paymentClient;
    public void checkout(Order order) {
        PaymentResponse resp = paymentClient.processPayment(new PaymentRequest(order));
    }
}
```

16. Secure Coding Practices (SAST / DAST)

Q: What is the difference between SAST and DAST?

A: SAST (Static Application Security Testing) analyzes source code, bytecode, or binaries without executing the application — catches issues like SQL injection patterns, hardcoded secrets, or insecure crypto usage early in development (e.g., SonarQube, Checkmarx, Fortify run in the CI pipeline on every commit/PR). DAST (Dynamic Application Security Testing) tests a running application from the outside, simulating real attacks against live endpoints (e.g., OWASP ZAP, Burp Suite) — catches runtime/config issues SAST can't see, like misconfigured headers or auth bypass in a deployed environment, but typically runs later (staging/pre-prod) since it needs a running app.

Q: How do you prevent SQL Injection in a Spring/JPA application?

A: Always use parameterized queries / prepared statements — never concatenate user input directly into a query string. Spring Data JPA's derived query methods and @Query with @Param are safe by default since they use bind parameters. The danger case is native/dynamic queries built with string concatenation — avoid that, or use JdbcTemplate/EntityManager's parameter-binding methods instead.

```
// VULNERABLE - never do this
String sql = "SELECT * FROM employee WHERE name = '" + userInput + "'";

// SAFE - parameterized
@Query("SELECT e FROM Employee e WHERE e.name = :name")
List<Employee> findByName(@Param("name") String name);
```

Q: What is the OWASP Top 10, and can you name a few relevant entries for a backend developer?

A: OWASP Top 10 is a regularly updated list of the most critical web application security risks. Ones most relevant to backend/API work: Injection (SQL/NoSQL/command injection), Broken Access Control (missing authorization checks on endpoints), Cryptographic Failures (weak/missing encryption of sensitive data), Security Misconfiguration (default credentials, verbose error messages leaking stack traces), and Vulnerable/Outdated Components (unpatched dependencies with known CVEs).

Q: How do you avoid leaking sensitive information (like stack traces or internal details) in API error responses?

A: Never return raw exception messages or stack traces directly to the client — use a global exception handler (@RestControllerAdvice) that maps internal exceptions to a generic, safe error response, while logging the full

detail server-side only. Also avoid verbose default error pages in production (server.error.include-stacktrace=never in Spring Boot).

Q: How would you securely store and manage secrets (DB passwords, API keys) instead of putting them in application.properties?

A: Avoid committing secrets to source control at all. Use a dedicated secrets manager (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) that the application fetches credentials from at startup/runtime, or inject them as environment variables from your deployment platform/CI-CD secrets store, keeping them out of the config repo entirely — this also ties back to why a Config Server repo itself shouldn't hold raw secrets, only non-sensitive config, with sensitive values encrypted or sourced from Vault instead.

17. Persistence Framework & Query Optimization (Advanced)

Q: What is batch processing in JPA/Hibernate, and why does it matter for bulk inserts/updates?

A: Without batching, Hibernate sends one SQL statement per entity, meaning inserting 10,000 rows means 10,000 separate round-trips to the database — very slow. Batching groups multiple statements into fewer round-trips using JDBC's batch execution. Configure via `hibernate.jdbc.batch_size`, and make sure entities use a sequence-based (not IDENTITY) ID generation strategy, since IDENTITY forces Hibernate to insert one row at a time to get the generated key back immediately.

```
# application.properties
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

Q: What is the difference between first-level and second-level cache in Hibernate?

A: First-level cache (persistence context / session) is enabled by default and scoped to a single transaction/session — repeated finds for the same entity within one transaction hit the cache, not the DB. Second-level cache is optional, shared across sessions/transactions (often backed by Ehcache or Redis), and caches entity data across multiple requests — useful for read-heavy, rarely-changing reference data, but risks staleness if not invalidated carefully on updates.

Q: How do you write a custom, optimized native query when JPQL/derived methods aren't efficient enough?

```
@Query(value = "SELECT * FROM employee e WHERE e.department_id = :deptId " +
               "ORDER BY e.salary DESC LIMIT :limit", nativeQuery = true)
List<Employee> findTopEarnersByDept(@Param("deptId") Long deptId, @Param("limit") int limit);
```

Q: What is optimistic locking, and how do you implement it with @Version?

A: Optimistic locking assumes conflicts are rare — instead of locking a row for the duration of a transaction (pessimistic locking), it checks at commit time whether the row changed since it was read, using a version column. If another transaction updated the row in between, an `OptimisticLockException` is thrown and the caller can retry. Good for low-contention scenarios; avoids the throughput cost of holding real database locks.

```
@Entity
public class Account {
    @Id Long id;
    double balance;
    @Version
    private Long version; // Hibernate auto-checks/increments this on update
}
```

Q: Pessimistic locking — when would you actually need it over optimistic?

A: Use pessimistic locking (SELECT ... FOR UPDATE, via @Lock(LockModeType.PESSIMISTIC_WRITE)) when conflicts are frequent and you want to guarantee no other transaction can even read-for-update the same row until you're done — e.g., decrementing limited inventory stock during a flash sale, where retrying repeatedly under optimistic locking would cause a storm of failed transactions.

18. CI/CD Pipeline

Q: Walk through a typical CI/CD pipeline for a Spring Boot microservice.

A: Developer pushes code to a feature branch -> CI server (Jenkins/GitHub Actions/GitLab CI) triggers on the PR: checkout -> build (Gradle/Maven) -> run unit tests -> run static analysis (SonarQube/SAST) -> build a Docker image -> push image to a registry (e.g., ECR/Docker Hub) tagged with a version/commit SHA -> deploy to a dev/staging environment automatically -> run integration/smoke tests there -> on merge to main and a manual approval gate, deploy to production, often via a blue-green or canary rollout strategy with automated rollback if health checks fail.

Q: What's the difference between Continuous Delivery and Continuous Deployment?

A: Continuous Delivery means every change that passes the pipeline is automatically packaged and ready to deploy, but an actual human approval/trigger is required for the production release. Continuous Deployment goes one step further — every change that passes all automated checks is deployed to production automatically, with no manual gate at all.

Q: What is a blue-green deployment, and how does it minimize downtime/risk?

A: Two identical production environments exist ('blue' = currently live, 'green' = new version). You deploy the new version fully to green while blue keeps serving all live traffic, run smoke tests against green, then switch the load balancer/router to send traffic to green all at once. If something's wrong, you switch back to blue instantly — near-zero downtime and a fast, low-risk rollback path.

Q: What is a canary deployment, and how does it differ from blue-green?

A: Canary gradually shifts a small percentage of live traffic (e.g., 5%) to the new version while most traffic still goes to the old one, monitoring error rates/latency on that small slice before gradually increasing it to 100%. Unlike blue-green's all-at-once switch, canary limits the blast radius of a bad release to a small fraction of users while still catching issues with real production traffic.

Q: How would you debug a deployment that succeeded in the pipeline but the service is failing health checks in production?

A: Check the application logs/centralized logging first for startup exceptions (bad config, missing environment variable, DB connection failure). Verify environment-specific config (secrets, DB URL, feature flags) actually got injected correctly for prod, since a common cause is a config value that worked in staging but wasn't set for prod. Check the health check endpoint itself (Actuator's /actuator/health) to see which downstream dependency it's reporting as DOWN — e.g., DB, message broker, or a downstream service dependency. Roll back immediately if it's customer-impacting while you investigate.

Q: What's the purpose of tagging and branching strategies (e.g., GitFlow, trunk-based development) in CI/CD?

A: They define how code moves from development to production in a controlled, traceable way. GitFlow uses long-lived develop/release/hotfix branches, good for scheduled releases but can create long-lived merge conflicts. Trunk-based development keeps everyone committing frequently to a single main branch behind feature flags, favoring continuous integration and faster releases, which pairs naturally with CI/CD and microservices. Tags mark specific commits as release points (e.g., v2.3.1), giving a clear, immutable reference for what's actually deployed where.

19. Architecture & Design (High-Level Design Talking Points)

Q: How would you explain your recent project's architecture in an interview, structurally?

A: A strong answer covers: the business problem/context in one sentence, the high-level component diagram (which services exist and how they talk to each other — sync REST vs async events), the data storage choices and why (e.g., relational for transactional order data, a cache for hot reads, maybe a search index), key non-functional requirements you had to design for (expected scale, latency targets, availability requirements), and one interesting trade-off or challenge you navigated (e.g., choosing eventual consistency via Saga over a distributed transaction).

Q: What are non-functional requirements (NFRs), and can you give examples relevant to a backend service?

A: NFRs describe how a system should behave, as opposed to what it should do (functional requirements). Examples: latency (95th percentile response time under 200ms), throughput (handle 1000 requests/sec), availability (99.9% uptime SLA), scalability (horizontal scaling under load), security (data encryption at rest/in transit), and maintainability (test coverage, documentation, deployability). Interviewers often probe whether you considered these in your project, not just whether the feature worked.

Q: What's the difference between horizontal and vertical scaling, and which do microservices typically favor?

A: Vertical scaling means adding more resources (CPU/RAM) to a single instance — simple but has a hard ceiling and a single point of failure. Horizontal scaling means adding more instances of a service behind a load balancer — microservices are specifically designed to favor this, since stateless services can be scaled out independently based on their individual load, which is much harder to do with a single large monolith.

Q: Pseudocode/talking point: how would you design a component diagram for an e-commerce checkout flow?

```
[API Gateway]
|
+--> [Order Service] --(sync REST)--> [Inventory Service] (check stock)
|
|           |
|           +--(publish OrderPlacedEvent)--> [Kafka]
|
+--> [Payment Service] <--(consumes OrderPlacedEvent)-- [Kafka]
|
|           |
|           +--(sync REST)--> [External Payment Gateway]
|
|           +--(publish PaymentCompletedEvent)--> [Kafka]
|
|           |
|           [Notification Service] consumes -> sends email/SMS
|           [Order Service] consumes -> updates order status to APPROVED
```

Q: What does 'separation of concerns' mean at an architectural level, and why does layering (Controller -> Service -> Repository) matter?

A: Each layer has one clear responsibility: Controller handles HTTP concerns (request/response mapping, status codes), Service holds business logic (validation rules, orchestration across repositories/other services), Repository handles persistence (queries, mapping to/from the database). This makes each layer independently testable (mock the service in a controller test, mock the repository in a service test), easier to reason about, and easier to change one layer (e.g., swap the DB) without rippling changes through the others.

20. Agile Process & Team Management

Q: Walk through a typical Scrum sprint cycle and its ceremonies.

A: Sprint Planning (team commits to a scoped set of backlog items for the sprint, usually 1-2 weeks), Daily Standup (each member shares what they did yesterday, what's next, any blockers — kept short, ~15 min), Backlog Refinement/Grooming (ongoing, breaking down and estimating upcoming stories), Sprint Review/Demo (showcase completed work to stakeholders at sprint end), and Sprint Retrospective (team reflects on what went well/poorly and commits to process improvements for the next sprint).

Q: What is story point estimation, and how does Planning Poker work?

A: Story points estimate relative effort/complexity/uncertainty of a task, not raw hours — often using a Fibonacci-like scale (1, 2, 3, 5, 8, 13...) since the gaps between larger numbers reflect growing uncertainty. In Planning Poker, each team member privately picks a card representing their estimate for a story, then everyone reveals simultaneously — big discrepancies trigger a discussion (the person with the lowest and highest estimates explains their reasoning) before re-voting, which surfaces hidden assumptions or missing requirements early.

Q: What's the difference between Scrum and Kanban?

A: Scrum works in fixed-length sprints with a set of committed stories per sprint and defined ceremonies. Kanban is a continuous flow model — no fixed sprints, work items move through columns (To Do -> In Progress -> Done) with WIP (work-in-progress) limits per column to prevent bottlenecks, and new work is pulled in as capacity frees up rather than planned in batches. Kanban suits teams with more unpredictable, ad hoc, or support-driven work; Scrum suits teams delivering discrete features on a predictable cadence.

Q: How do you handle scope creep or an urgent production issue interrupting a committed sprint?

A: Raise it transparently with the team/Scrum Master rather than silently absorbing it — decide together whether the urgent item genuinely needs to displace committed sprint work (e.g., a production incident) versus can wait for the next sprint, and if something is pulled in, something of equivalent size should typically come out of the sprint commitment to keep the sprint goal realistic and avoid burnout/quality shortcuts.

21. React Fundamentals (Hands-On Basics + Code)

» TIP: Given limited hands-on React depth, focus on being genuinely solid on this section rather than skimming everything. If asked about something you haven't used (e.g., Redux-Saga), it's fine to say so honestly and pivot to what you have used — that's a stronger answer than guessing.

Q: What is JSX, and why can't the browser run it directly?

A: JSX is a syntax extension that lets you write HTML-like markup directly inside JavaScript. Browsers can't execute it natively — a build tool (Babel, part of the React build pipeline) transpiles JSX into plain `React.createElement(...)` calls before the code ships to the browser.

```
// JSX
function Greeting() {
  return <h1>Hello, {userName}!</h1>;
}

// What it compiles to under the hood
function Greeting() {
  return React.createElement('h1', null, 'Hello, ', userName, '!');
}
```

Q: Functional components vs class components — what's the modern default, and why?

A: Functional components (plain JavaScript functions returning JSX) are the modern standard since Hooks (React 16.8) let them manage state and side effects, which previously required a class component. Class components use lifecycle methods (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`) and

'this.state'/this.setState()'. New code is almost always written as functional components with hooks; class components mostly appear in legacy codebases now.

```
// Functional component with hooks (modern standard)
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;
}

// Equivalent class component (legacy style)
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }
  render() {
    return (
      <button onClick={() => this.setState({ count: this.state.count + 1 })}>
        Count: {this.state.count}
      </button>
    );
  }
}
```

Q: Explain useState and useEffect — the two hooks you'll use constantly.

A: useState(initialValue) returns a [value, setter] pair; calling the setter schedules a re-render with the new value. useEffect(callback, dependencyArray) runs the callback after render, and re-runs it only when something in the dependency array changes — used for side effects like fetching data, subscribing to events, or manually touching the DOM. An empty dependency array ([]) means 'run once, on mount only'; omitting the array entirely means 'run after every render' (usually not what you want).

```
function EmployeeList() {
  const [employees, setEmployees] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/employees')
      .then(res => res.json())
      .then(data => {
        setEmployees(data);
        setLoading(false);
      });
  }, []); // empty array = fetch once when component mounts

  if (loading) return <p>Loading...</p>;
  return (
    <ul>
      {employees.map(emp => <li key={emp.id}>{emp.name}</li>)}
    </ul>
  );
}
```

Q: What is the Virtual DOM, and why does it make React fast?

A: The Virtual DOM is a lightweight in-memory representation of the actual DOM. When state changes, React first updates this virtual copy, then compares ('diffs') it against the previous virtual version to compute the minimal set of actual DOM changes needed, and only applies that minimal patch to the real DOM. Direct DOM manipulation is expensive (triggers layout/reflow), so batching and minimizing real DOM writes this way is what gives React its performance advantage over manually updating the DOM for every small state change.

Q: What are props, and how do they differ from state?

A: Props are read-only data passed from a parent component down to a child — the child cannot modify its own props directly. State is data owned and managed internally by a component itself, which can change over time and triggers a re-render when updated via its setter. A common pattern: state lives in a parent component and is passed down as props to children, with callback functions also passed down as props so children can request changes.

```
function ParentList() {
  const [items, setItems] = useState(['Apple', 'Banana']);

  function removeItem(index) {
    setItems(items.filter((_, i) => i !== index));
  }

  return (
    <ul>
      {items.map((item, i) => (
        <ChildItem key={i} name={item} onRemove={() => removeItem(i)} />
      ))}
    </ul>
  );
}

function ChildItem({ name, onRemove }) { // receives props, can't modify them directly
  return <li>{name} <button onClick={onRemove}>Remove</button></li>;
}
```

Q: Controlled vs uncontrolled components — explain with a form input example.

A: A controlled component's value is fully driven by React state — the input's value comes from state, and every keystroke updates that state via onChange, so React is the 'single source of truth'. An uncontrolled component lets the DOM itself hold the value, and you read it only when needed (e.g., on submit) via a ref, rather than tracking every keystroke in state.

```
// Controlled
function ControlledInput() {
  const [name, setName] = useState('');
  return <input value={name} onChange={e => setName(e.target.value)} />;
}

// Uncontrolled
function UncontrolledInput() {
  const inputRef = useRef();
  function handleSubmit() {
    alert(inputRef.current.value); // read the value only on submit
  }
  return (
    <>
      <input ref={inputRef} defaultValue="" />
      <button onClick={handleSubmit}>Submit</button>
    </>
  );
}
```

Q: What is the Context API, and what problem does it solve (vs passing props down manually)?

A: Context lets you share a value (e.g., the logged-in user, or a theme) across many components without manually passing it down as a prop through every intermediate level ('prop drilling'). A Provider at the top supplies the value, and any descendant component can read it directly via useContext, no matter how deeply nested.

```

const UserContext = React.createContext(null);

function App() {
  const [user, setUser] = useState({ name: "Alice" });
  return (
    <UserContext.Provider value={user}>
      <Dashboard />
    </UserContext.Provider>
  );
}

function Dashboard() {
  return <ProfileBadge />; // user isn't passed as a prop through here
}

function ProfileBadge() {
  const user = useContext(UserContext); // reads directly, no prop drilling
  return <span>Welcome, {user.name}</span>;
}

```

Q: Why does a list rendered with .map() need a unique 'key' prop, and what happens if you use the array index as the key?

A: The key helps React's diffing algorithm identify which items changed, were added, or were removed between re-renders, so it can update the DOM efficiently and preserve component state correctly per item. Using the array index as a key can cause bugs when the list is reordered, filtered, or has items inserted/removed in the middle — React may match the wrong item to the wrong DOM node/state. Prefer a stable, unique ID from your data instead of the index whenever the list can change order or size.

Q: What is React Router used for, and how do you define basic routes?

A: React Router handles client-side routing in a single-page app — mapping different URL paths to different components without a full page reload, and managing browser history/back-forward navigation.

```

import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/employees">Employees</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/employees" element={<EmployeeList />} />
        <Route path="/employees/:id" element={<EmployeeDetail />} />
      </Routes>
    </BrowserRouter>
  );
}

```

Q: What problem does Redux (or a state management library) solve that useState/Context don't handle well?

A: As an app grows, state needed by many unrelated components across the tree becomes hard to manage with just local useState, and Context re-renders every consumer whenever the context value changes, which doesn't scale well for frequently-changing, app-wide state. Redux centralizes all application state in a single store with a strict, predictable update pattern: components dispatch actions, a reducer function computes the new state from the current state and the action, and components subscribe to only the specific slices of state they need.

```
// A simple reducer
function counterReducer(state = { count: 0 }, action) {
  switch (action.type) {
    case 'INCREMENT': return { count: state.count + 1 };
    case 'DECREMENT': return { count: state.count - 1 };
    default: return state;
  }
}

// Using it in a component (with react-redux)
function Counter() {
  const count = useSelector(state => state.count);
  const dispatch = useDispatch();
  return <button onClick={() => dispatch({ type: 'INCREMENT' })}>{count}</button>;
}
```

Q: What is Redux-Thunk used for?

A: Plain Redux actions must be plain objects and reducers must be pure functions — neither can directly handle asynchronous logic like an API call. Redux-Thunk lets an action creator return a function instead of a plain object; that function receives dispatch and can perform async work (like fetching data), dispatching further actions (e.g., a loading action, then a success or failure action) once the async operation resolves.

```
function fetchEmployees() {
  return async function (dispatch) {
    dispatch({ type: 'FETCH_START' });
    try {
      const res = await fetch('/api/employees');
      const data = await res.json();
      dispatch({ type: 'FETCH_SUCCESS', payload: data });
    } catch (err) {
      dispatch({ type: 'FETCH_ERROR', error: err.message });
    }
  };
}
```

Q: What's the honest, high-level difference between Redux-Thunk and Redux-Saga (fine to mention even without deep hands-on)?

A: Both handle async side effects in Redux, but Thunk uses simple functions/promises and is easier to learn/read for straightforward async flows. Saga uses generator functions and a more powerful effects model, better suited for complex async workflows — cancellable requests, debouncing, coordinating multiple async actions, or retry logic — at the cost of a steeper learning curve.

Q: What does npm/package.json actually manage, at a basic level, for a React project?

A: package.json declares the project's dependencies (React itself, React Router, any UI library) and devDependencies (build tools, testing libraries), pinned to specific version ranges, plus scripts (like 'start', 'build', 'test') that npm/yarn run. Running npm install reads this file and downloads the exact dependency tree into node_modules, so the project is reproducible across machines/CI.

Q: At a conceptual level, what does a build tool like Webpack do for a React app?

A: It bundles many separate JS/CSS/asset files (and JSX, which needs transpiling via Babel first) into a small number of optimized output files the browser can load efficiently — handling module resolution (import/export), code splitting (so users don't download code for pages they haven't visited yet), minification, and often a dev server with hot-reloading during development.

Q: What is shouldComponentUpdate / React.memo, and when would you use it?

A: Both let you skip an unnecessary re-render of a component when its props haven't meaningfully changed — shouldComponentUpdate is the class-component lifecycle method for this, and React.memo is the

functional-component equivalent (a higher-order wrapper that does a shallow prop comparison before re-rendering). Useful as a performance optimization for components that render often or are expensive to render, but shouldn't be reached for by default — premature memoization can add complexity without measurable benefit.

```
const EmployeeRow = React.memo(function EmployeeRow({ employee }) {  
  console.log("Rendering", employee.name);  
  return <li>{employee.name}</li>;  
});  
// Only re-renders if the `employee` prop actually changes (shallow comparison)
```

22. Final Skim Before the Call (Rounds 2 & 3 Combined)

- @Transactional is proxy-based: doesn't work on private methods or on self-invocation within the same class.
- Circuit Breaker states: CLOSED -> OPEN (fail fast) -> HALF_OPEN (test recovery) -> CLOSED.
- Saga = eventual consistency + compensating transactions, because no single DB spans all microservices.
- N+1 problem: fix lazy-loaded collections with JOIN FETCH or @EntityGraph.
- POST = create, not idempotent. PUT = replace, idempotent. PATCH = partial update.
- 201 Created for POST, 204 No Content for DELETE/no-body updates, 409 Conflict for duplicate/state conflicts.
- @RestControllerAdvice + @ExceptionHandler = centralized, JSON-serialized error handling.
- Correlation IDs + centralized logging (ELK) are how you trace one request across many microservices.
- Collectors.toMap needs a 3-arg merge function whenever API data might have duplicate keys.
- RestTemplate.getForObject / WebClient for calling external JSON APIs; ObjectMapper/JsonNode when there's no POJO.
- Always wrap external API calls in try/catch for HttpClientErrorException and set explicit timeouts.
- Practice LRU cache (LinkedHashMap access-order) and a simple rate limiter - both are common coding-round asks.
- SAST = scans code before it runs (SonarQube); DAST = attacks a running app (OWASP ZAP). Know the difference cold.
- Never build SQL via string concatenation - always parameterized queries / @Param bindings.
- Optimistic locking (@Version) for low-contention; pessimistic locking (SELECT FOR UPDATE) for high-contention writes.
- Blue-green = instant full switch with instant rollback. Canary = gradual traffic shift, smaller blast radius.
- Be ready to whiteboard your project's component diagram and name at least one real NFR (latency/scale/availability) you designed for.
- React: useState = local reactive value, useEffect = side effects tied to a dependency array, Context = avoids prop drilling.
- Controlled inputs = React state is source of truth (onChange). Uncontrolled = DOM holds value, read via ref.
- Never use array index as a React list key if the list can reorder, filter, or have items inserted/removed.
- For React gaps in your experience, say so plainly and pivot to what you've actually built - it reads far better than guessing.
- Be ready to draw your own project's microservice architecture on a virtual whiteboard - practice narrating it out loud.

- If asked about a project, mention concrete numbers/scale (requests/sec, data size) if you can - it signals real experience.

Two rounds down the middle, one to go - you've built real depth. Go get it.